

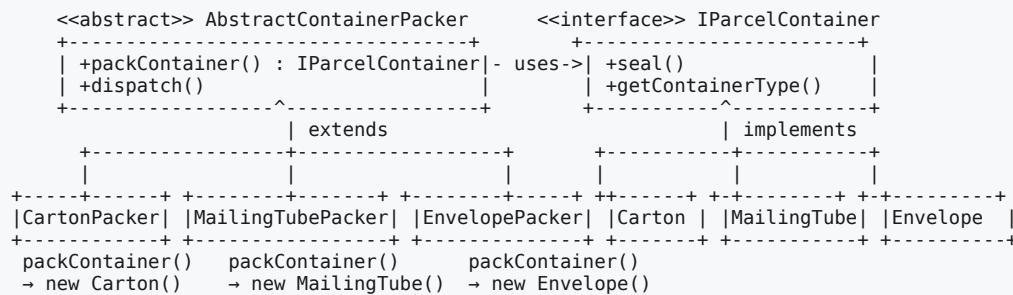
Factory Method Design Pattern

Intent

Define an interface for creating an object, but let **subclasses decide which concrete class to instantiate**. The creator's algorithm (`dispatch()`) works against the abstract product (`IParcelContainer`); the *factory method* (`packContainer()`) is the single point each subclass overrides to supply its own product.

This is the **canonical GoF Factory Method**: an abstract `AbstractContainerPacker` with one abstract creation step, and one concrete creator per product. The client picks a packer, never a container — it calls `new` on `CartonPacker`, but never on `Carton`.

UML class diagram



The players

<code>enums/ContainerType</code> <code>ENVELOPE)</code>	each product's self-declared identity (<code>CARTON</code> , <code>MAILING_TUBE</code> ,
<code>factory/contract/IParcelContainer</code>	the product interface
<code>factory/contract/concrete/Carton</code> <code>/Envelope</code> <code>/MailingTube</code>	concrete products
<code>factory/AbstractContainerPacker</code>	the abstract creator – declares <code>packContainer()</code> , owns <code>dispatch()</code>
<code>factory/creators/CartonPacker</code> <code>/EnvelopePacker</code> <code>/MailingTubePacker</code>	concrete creators – one per product, each overrides <code>packContainer()</code>

The code, line by line

`IParcelContainer` — the product interface

```
public interface IParcelContainer {
    ContainerType getContainerType();
    void seal();
}
```

- `seal()` is the behavior every product offers; it's what the creator's `dispatch()` ultimately calls.
- `getContainerType()` lets each product declare its own identity (`Carton` says `CARTON`), so nothing outside the product needs a mapping table.

`Carton` / `Envelope` / `MailingTube` — concrete products

```
public class Carton implements IParcelContainer {
    @Override public void seal() { System.out.println("Carton sealed with packing tape."); }
    @Override public ContainerType getContainerType() { return ContainerType.CARTON; }
}
```

- Plain classes — the pattern needs nothing more. Only the concrete *creator* knows which one gets instantiated.

AbstractContainerPacker — the abstract creator

```
public abstract class AbstractContainerPacker {  
    public abstract IParcelContainer packContainer();  
  
    public void dispatch() {  
        IParcelContainer container = packContainer();  
        container.seal();  
    }  
}
```

- `packContainer()` **is the factory method** — the one deferred decision. It returns the abstract `IParcelContainer`, so nothing in this class ever names a concrete product.
- `dispatch()` is the **template step** that makes the pattern useful: it contains the creator's real algorithm (pack, then seal). Subclasses inherit the algorithm and customize only the creation step. If `dispatch()` grew (validate → pack → seal → label → log), every packer would gain those steps for free.

CartonPacker / EnvelopePacker / MailingTubePacker — concrete creators

```
public class CartonPacker extends AbstractContainerPacker {  
    @Override  
    public IParcelContainer packContainer() {  
        return new Carton();  
    }  
}
```

- One line of real logic: `new Carton()`. This is the **only place in the module where a concrete product is constructed**. The other two packers are identical except for the product they `new`.
- The return type stays `IParcelContainer` (not `Carton`) — callers get the abstraction even from the concrete creator.

Why the design decisions

Why an abstract class + subclasses instead of one factory with a `switch`/map?

Because the *variation point is the creation itself*, and Factory Method places that variation in the type system:

- Adding `Crate` = add `Crate` implements `IParcelContainer` + `CratePacker` extends `AbstractContainerPacker`. **No existing file is edited** — Open/Closed by construction, no central registry to grow.
- Each packer can later specialize more than construction (pick a tape gauge, pre-print a shipping label) without touching its siblings.
- A key-indexed factory (enum → instance map) answers a different question — "give me the product for this key at runtime." That variant belongs to a *parameterized* factory; this module demonstrates the GoF subclassing form.

Why does `dispatch()` live in the abstract creator?

That's the point of the pattern: **the creator is not just a maker, it's a user of its own product**. `dispatch()` is code written once against `IParcelContainer` that works for every current and future subclass. The factory method exists so this shared algorithm can create the right product without knowing its class.

Why does `packContainer()` return `IParcelContainer` and not the concrete type?

So the inherited `dispatch()` — and any caller — stays coupled only to the abstraction. The concrete class name appears exactly once, inside its packer.

Why do products still carry `getContainerType()`?

Each product self-describes its identity. Nothing in the packer flow needs it, but it keeps a product's key with the product itself — the single source of truth if a lookup table (or serialization) is ever layered on top.

Execution flow (as run from `main`)

```
PackagingLineApplication.main  
├── SpringApplication.run(...) boots the app (the pattern itself doesn't need Spring)
```

```

└─ List.of(new CartonPacker(), new MailingTubePacker(), new EnvelopePacker())
    └─ the client chooses PACKERS, never products
└─ packers.forEach(AbstractContainerPacker::dispatch)
    └─ CartonPacker.dispatch()
        └─ packContainer() → new Carton() → seal() → "Carton sealed with packing tape."
    └─ MailingTubePacker.dispatch()
        └─ packContainer() → new MailingTube() → seal() → "Mailing tube sealed with end
caps"
    └─ EnvelopePacker.dispatch()
        └─ packContainer() → new Envelope() → seal() → "Envelope sealed with adhesive
flap"

```

Expected output

Captured from an actual run:

```

Carton sealed with packing tape.
Mailing tube sealed with end caps
Envelope sealed with adhesive flap

```

(Spring Boot banner, Tomcat startup, and logging lines precede this — the module pulls in `spring-boot-starter-web`, so an embedded Tomcat starts on port 8080 even though the pattern itself needs no web layer. The JVM keeps running to serve HTTP after these three lines print; stop it with Ctrl-C or a timeout.)

How to run

```

# From the module root directory
JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 mvn -o clean compile

JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 java -cp target/classes:$(mvn -o dependency:build-classpath -
Dmdep.outputFile=/dev/stdout -q) \
  com.design.patterns.PackagingLineApplication

```

The `-o` (offline) flag works once the parent reactor and dependencies are already in your local Maven cache; drop it on a first build.

Factory Method vs. Abstract Factory (so you don't confuse them)

- **Factory Method (this module)** — one product per creator, and the *subclass* is the decision. You pick the packer (`CartonPacker`), and it makes its one product.
- **Abstract Factory** (the sibling module) — a **matched set** of related products (a chair *and* a table that belong together) behind one factory interface.

Rule of thumb: if choosing the factory chooses one product, it's Factory Method; if choosing the factory chooses a whole family, it's Abstract Factory.